BROADWAY INFOSYS
Since 2008

CERTIFIED ISO 9001:2015 COMPANY

Building Global IT Professionals since 2008

19K+
Certified Students

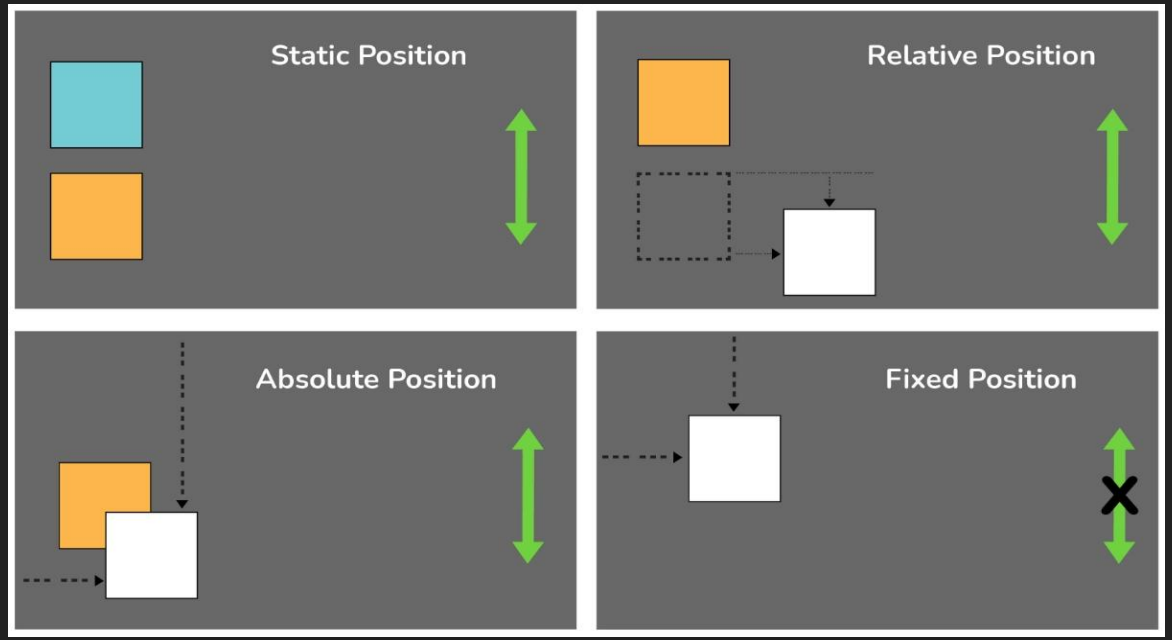
160+
Courses

85+
Qualified Teachers

www.broadwayinfosys.com

CSS Position Layout

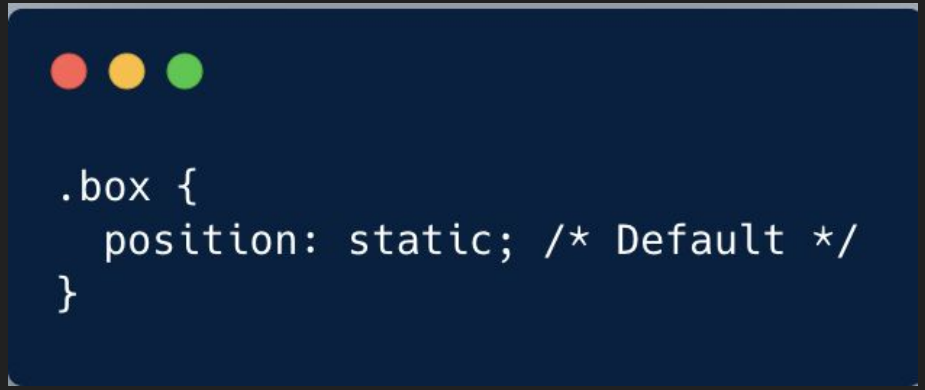
The position property specifies the positioning type for an element.



Elements are positioned to their final location with the top, bottom, left, and right properties.

The Normal Document Flow —- Static

By default, every HTML element uses Static Positioning.
This is the browser's "automatic" behavior. Elements follow the natural document flow, stacking from top-to-bottom like physical bricks in a wall.

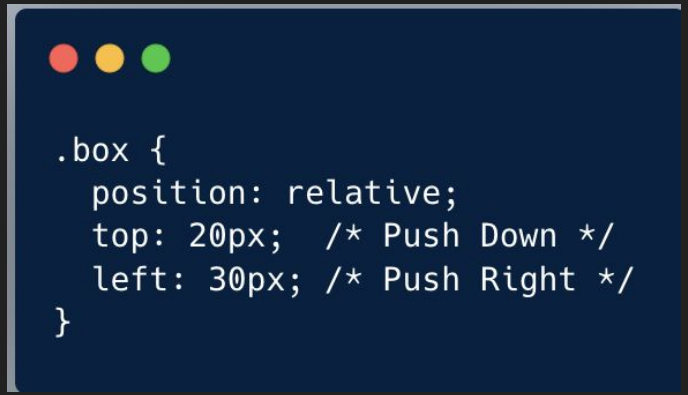


```
.box {
  position: static; /* Default */
}
```

Positional offset properties (top, left, right, bottom) have zero effect on static elements.

Position : Relative

Relative Positioning allows you to "nudge" an element away from its natural spot. However,even though the element moves visually, the browser still acts as if it is sitting in its original position.

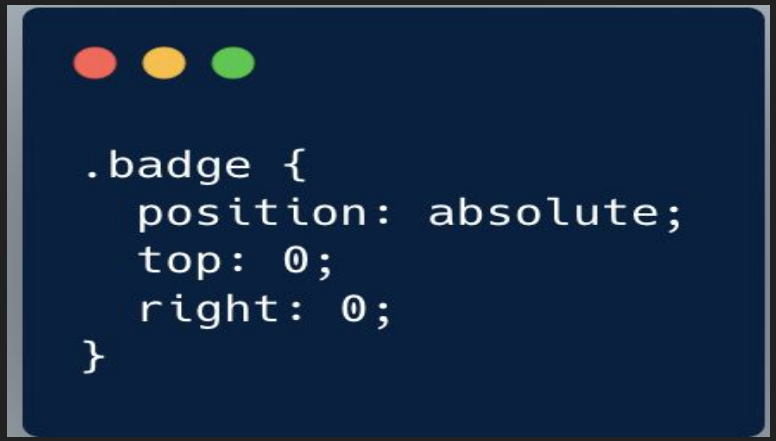


```
.box {
  position: relative;
  top: 20px;  /* Push Down */
  left: 30px; /* Push Right */
}
```

The box moves, but the gap where it used to be stays reserved.
No other element can take that space.

Position: Absolute

Absolute Positioning completely removes an element from the normal document flow



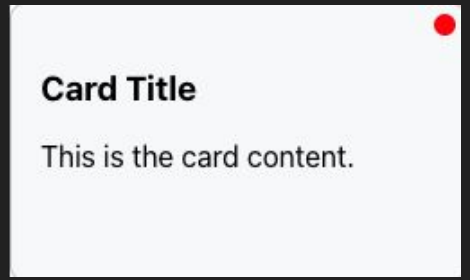
```
.badge {
  position: absolute;
  top: 0;
  right: 0;
}
```

The element flies out of the layout. It floats on its own layer, overlapping other elements.

The Power Duo: Relative + Absolute

Use them together to lock a child element inside a parent box.

- Parent (Relative): The Anchor/Container.
- Child (Absolute): The Sticker inside the container.




```css
.card { position: relative; }

.dot {
  position: absolute;
  top: 5px; right: 5px;
}
```

Stacking & Overflow

When elements start moving out of flow, they inevitably overlap. We use Z-index to handle who stacks on top, and Overflow to safely manage text that spills out of small containers.

- Z-index: Controls the 3D depth layers. Higher numbers render on top of lower numbers (Requires an explicit non-static position property to function).
- Overflow: Handles structural spillage. hidden cleanly crops off content passing the border; scroll forces inner navigation scrollbars.

```
.modal { z-index: 999; }
```

```
.text-box {
  width: 200px;
  height: 100px;
  overflow-y: scroll;
}
```

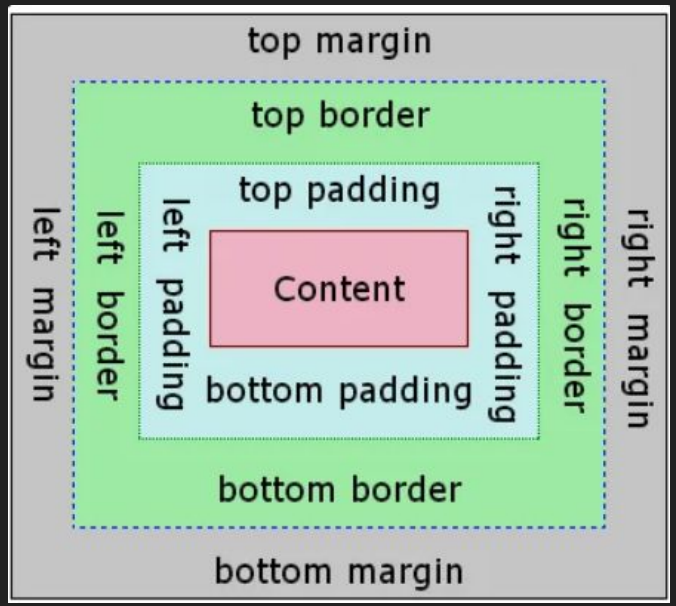
Practice Task 1: Positioning

https://github.com/uzalgunner/demo-class

The Foundation: The Box Model

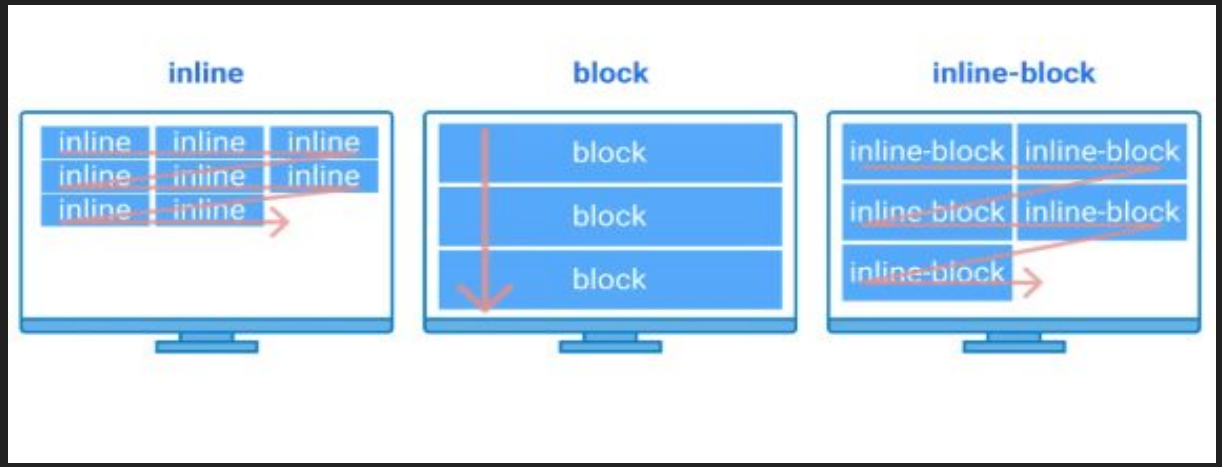
- Padding: The clear internal breathing room surrounding the content *inside* the border edge.
- Border: The physical structural frame outlining the padding space.
- Margin: The clear external safety buffer zone separating this element *outside* from neighboring boxes.

```css
.box {
  padding: 20px;
  border: 2px solid white;
  margin: 30px;
}
```



Display Property: Block vs. Inline

- Block Display : takes full width. Starts a new line (e.g. div, h1).
- Inline Display : takes only needed width. No new line (e.g. span, a).
- Inline-Block : best of both! Stays in line but allows width/height styling.



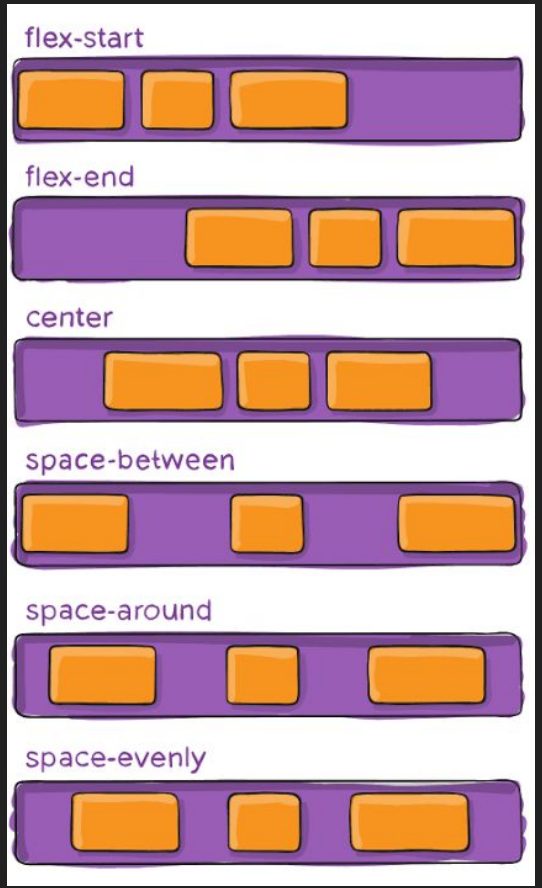
Flexbox Engine (1D Layout)

Flexbox is a CSS layout system that makes it easy to arrange, align, and distribute items in a row or column.

The idea is simple:

- Parent → becomes a flex container.
- Children → become flex items.

```css
.navbar {
  display: flex;
  flex-direction: row;
}
```



Flexbox Alignment

The true strength of the Flexbox layout engine is the ability to align, distribute, and center child boxes perfectly with simple keyword commands, entirely replacing manual calculation math

- **justify-content: Align along main axis (horizontal)**
- **align-items: align along cross axis (vertical).**

```
.center-box {
  display: flex;
  justify-content: center;
  align-items: center;
}
```

CSS Grid Engine (2D Layout)

While Flexbox works beautifully for elements oriented along a single direction line, CSS Grid is a powerful two-dimensional layout matrix.

It handles rows (Y-axis) and columns (X-axis) simultaneously.

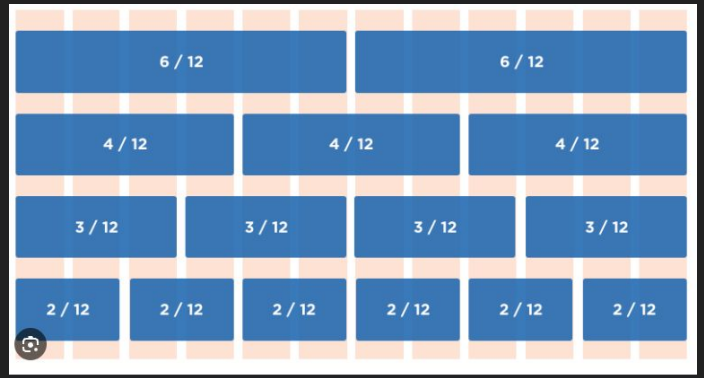
```css
.gallery {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 20px;
}
```

- **display: grid; Set on the parent frame to create a structured grid system matrix layout.**
- **grid-template-columns: Explicitly defines the structural column track widths (e.g., 1fr represents one proportional fraction share of free space).**
- **gap: Sets clean gutter channels directly between the rows and columns, eliminating the need for complex, messy element margin math.**

The Professional 12-Column Grid

A full-width header spans all 12 layout columns.

```css
.grid-wrapper {
  display: grid;
  grid-template-columns: repeat(12, 1fr);
}
```



Practice Task 2 : Flex and Grid

THANK YOU!

BROADWAY
INFOSYS
Since 2008

Shree Ganesh Marg, Subidhanagar, Tinkune, Kathmandu
01-4117578 / 4111849 / 9841002000
www.broadwayinfosys.com